



Agent-Driven Workflow

Playbook SDD + ADD | Chương 10

Từ Intent Đến Delivery — ADD trong thực chiến

6

Sections

4

Pha ADD

3

Constraint Layers

"ADD không làm bạn code chậm hơn. Nó làm bạn rework ít hơn."

Giới thiệu chương

Chương 10 là gì?

Chương 9 đã giải phẫu Agent từ bên trong. Chương này đặt Agent vào guồng làm việc thực tế: ADD (Agent-Driven Development) — một phương pháp luận đặt Agent làm người thực thi chính trong vòng lặp phát triển, với con người đóng vai trò Director (định hướng và phê duyệt), không phải Executor (gõ code).

ADD vs SDD — Mối quan hệ cốt lõi

SDD (Chương 5–8)

Đảm bảo bạn biết WHAT để build trước khi AI build.

ADD (Chương 10)

Đảm bảo AI execute HOW một cách hiệu quả nhất.

Kết hợp

SDD cung cấp Spec → ADD cung cấp Execution Workflow.

ADD không thay thế SDD

Chúng hoạt động cùng nhau.

Cảnh báo

ADD mà không có Spec tốt = fast execution of wrong thing.

Tỷ lệ 30/70

- 30% lý thuyết đặt nền tảng tư duy
- 70% thực hành — bài lab đối chứng Section 10.5
- Học bằng cách làm, không chỉ đọc
- Bài lab thuyết phục bằng trải nghiệm — không phải lời giải thích

Vai trò con người trong ADD

- Director — định hướng và phê duyệt
- Human time: ~20% (setup + review)
- AI time: ~80% (execute)
- Human gate tại các điểm rủi ro cao

Table of Contents

Mục lục Chương 10

10.1	Quy trình 4 pha của ADD	Context Setup · Intent · Execution · Human Review
10.2	Prompt Engineering cho Agentic Coding	WHAT không HOW · DoD · Templates · Good vs Bad
10.3	Task-Based Execution Model	Context Pollution · Plan-Act-Check · Fresh Session
10.4	Constraint Documents	3-layer Hierarchy · Global · Business · Safety
10.5	Hands-on Lab — Có vs. Không có ADD	Round 1 (No ADD) · Round 2 (With ADD) · Reflection
10.6	Kỹ thuật Nâng cao	Shadowing · Token Management · .agentignore

10.1 — Section 10.1

Quy trình 4 pha của ADD

ADD pipeline phản ánh triết lý cốt lõi: con người định nghĩa context và intent một lần, agent thực thi nhiều lần với human approval tại các điểm rủi ro cao.

1

Pha 1: Context Setup

Agent cần "biết mình là ai"

2

Pha 2: Intent Communication

WHAT, không phải HOW

3

Pha 3: Agentic Execution

+ Anti-patterns

4

Pha 4: Human Review

Duyệt Kế hoạch trước Code

❏ "Thiết lập tốt ở Pha 1 = ít can thiệp ở Pha 3" — đây là ROI của upfront investment.

10.1 — ADD Pipeline

ADD Pipeline — 4 Phases

ADD PIPELINE

PHA 1: CONTEXT SETUP (Một lần, đầu dự án)

AGENTS.md CLAUDE.md **Constraint** Docs

(Agent persona) (Project ctx) (Stack, rules, glossary)

Agent biết: Tôi là ai? Dự án này là gì? Giới hạn nào?

↓ (per-feature)

PHA 2: INTENT COMMUNICATION

User Story → Prompt → Definition **of** Done

"WHAT" không phải "HOW"

↓

PHA 3: AGENTIC EXECUTION

Agent tự: Plan → Code → Test → Fix → Iterate

Human gates: Approve Plan + Approve risky file changes

↓

PHA 4: HUMAN REVIEW & ITERATE

Review Plan (trước code) → Review Code → **Merge**/Iterate

Human **time**: ~20% (setup + review) | AI **time**: ~80% (**execute**)

10.1.1 — Pha 1

Pha 1 — Context Setup: Agent cần "biết mình là ai"

Đây là pha quan trọng nhất nhưng thường bị xem nhẹ nhất. Nhiều developer skip pha này và đi thẳng vào prompt. Kết quả: agent code "đúng logic" nhưng dùng sai framework, vi phạm conventions, tạo files sai cấu trúc, hay đặt tên sai style. Mọi thứ phải sửa lại.

⚠ Agent không có common sense về dự án của bạn. Những điều bạn "đương nhiên biết" — dùng Go không phải Python, snake_case cho file names, no third-party auth libs — agent không biết trừ khi được nói. AGENTS.md + CLAUDE.md là kênh truyền tải "common sense" đó.

AGENTS.md — "Persona" của Agent

- Định nghĩa seniority level của agent
- Area of expertise, coding philosophy
- Decision-making style
- Tools được phép dùng
- Các hành động bị cấm hoàn toàn

CLAUDE.md — "Project DNA"

- Lịch sử và kiến trúc dự án
- Các quyết định quan trọng đã được đưa ra
- Lesson learned từ incidents
- TL;DR 30 giây của toàn dự án
- Current sprint focus

AGENTS.md — Go Developer Persona

AGENTS.md — Go Microservice Project

AGENTS.md — Go Microservice Project

Version: 1.0.0 | Owner: @tech-lead

PERSONA

Bạn là Senior Go Developer với 7+ năm kinh nghiệm.

Philosophy: simplicity over cleverness, explicit over implicit.

Ưu tiên: correctness > performance > readability > terseness.

Câu hỏi trước khi code: "Có cách đơn giản hơn không?"

EXPERTISE

- Primary: Go 1.23+, PostgreSQL, Redis, gRPC, Kafka
- Secondary: Docker, Kubernetes, Prometheus

CODING PHILOSOPHY

- Error handling: explicit return errors, không panic trừ init
- Interfaces: define ở nơi dùng, không ở nơi implement
- Comments: explain WHY, không WHAT; tiếng Anh cho code comments

DECISION RULES

- Không chắc về architecture → hỏi, không assume
- Thấy violation của constraint → báo cáo, không workaround

TOOLS BẠN ĐƯỢC PHÉP DÙNG

- Read/write files trong: /src, /tests, /docs, /scripts
- Execute: go test, go build, go vet, gofmt, golangci-lint, make
- Git: status, diff, add, commit (không push, không force)

KHÔNG ĐƯỢC PHÉP

- Không xóa files mà không confirm với user
- Không thêm dependency vào go.mod mà không hỏi
- Không commit vào main/master trực tiếp
- Không bỏ qua existing tests khi refactor

CLAUDE.md — Project DNA

CLAUDE.md cung cấp "DNA" của dự án: lịch sử, kiến trúc, các quyết định quan trọng, và những "lesson learned" mà team đã học được qua đau thương. Giúp agent hiểu TẠI SAO dự án được structure như vậy — không chỉ biết nó trông như thế nào.

```
CLAUDE.md — OrderService (Go Microservice)

# CLAUDE.md — OrderService (Go Microservice)
# Version: 2.1 | Sprint: Q1-2025

## TL;DR (30 giây)
Order management microservice. REST API + gRPC.
Nhận events từ cart-service, emit events đến payment-service.
Database: PostgreSQL (read-replica cho reporting).

## KIẾN TRÚC
Clean Architecture: domain/ → usecase/ → interface/ → infra/
Domain logic KHÔNG được import infra packages.

## FILE STRUCTURE
/src
  /domain # Entities, Value Objects, Interfaces
  /usecase # Business logic, interactors
  /interface # HTTP handlers, gRPC servers, Kafka consumers
  /infra # DB, cache, external APIs
/tests
  /unit # No DB, no network (< 50ms)
  /integration # Docker required (run: make test-integration)

## QUAN TRỌNG — LESSON LEARNED
- [2024-11] KHÔNG dùng database transactions spanning > 1 service.
  Incident: deadlock khi cart + order đều lock inventory row.
  Fix: Saga pattern với compensation events.

- [2024-12] Error wrapping bắt buộc: fmt.Errorf("context: %w", err)
  Không dùng errors.New khi wrap existing error.

## CURRENT SPRINT FOCUS
Implementing bulk order API (SPEC: .sdd/features/feat-bulk-order/)
In-progress: T003 (OrderValidator) — xem TASKS.md
```

10.1.1 — Checklist

Checklist — Definition of Done cho Context Setup

☐ AGENTS.md: Persona được định nghĩa (seniority, expertise, philosophy)

☐ AGENTS.md: Allowed/forbidden actions được liệt kê rõ ràng

☐ CLAUDE.md: Architecture tóm tắt và file structure được document

☐ CLAUDE.md: Lesson learned từ incidents quan trọng được note

☐ Constitution.md: Hard rules (security, architecture) đã có

☐ Constraint docs: Stack, naming, patterns đã defined

☐ Chạy test: nhờ agent describe dự án lại = "đọc phòng" test

☐ "Thiết lập tốt ở Pha 1 = ít can thiệp ở Pha 3." Mỗi phút setup context tiết kiệm 10 phút debug sau.

Sau khi setup xong, hãy hỏi agent: *"Describe lại dự án này cho tôi nghe."* Nếu agent mô tả đúng architecture, stack, và constraints — Pha 1 thành công. Nếu agent confused hoặc mô tả sai — cần bổ sung thêm context.

10.1.2 — Pha 2

Pha 2 — Intent Communication: WHAT, không phải HOW

Pha này là nơi nhiều developer "overthink" hoặc "underthink". Overthinking: viết prompt chi tiết cách implement — rốt cuộc bạn đang viết pseudo-code thay vì spec. Underthinking: "làm tính năng login đi" — agent không có đủ context để làm đúng.

Nguyên tắc vàng: Mô tả WHAT bạn muốn đạt được và WHY, để agent tự quyết định HOW trong khuôn khổ constraints đã có. Điều này không mâu thuẫn với SDD — nếu có SPEC.md, intent chỉ là "implement spec này".

✗ Overthinking (HOW)

"Tao struct OrderValidator với method Validate(order Order) error"
→ bạn đang implement, không phải agent → mất đi autonomous reasoning của agent.

✓ WHAT + DoD (tốt nhất)

"Implement User registration. Done khi: (1) password hashed, (2) tests pass, (3) no linting errors" — rõ ràng, có thể verify, agent tự chọn HOW.

10.1.3 — Pha 3

Pha 3 — Agentic Execution: Agent làm việc, Human gates

Trong Pha 3, agent chạy Plan → Execute → Test loop tự động. Vai trò của bạn là gatekeeper tại hai điểm: (1) approve execution plan trước khi agent bắt đầu code, và (2) review file changes trước khi commit. Không can thiệp vào giữa quá trình trừ khi thấy red flags.



Anti-pattern: Micromanagement trong Pha 3

- ❌ Micromanagement phá vỡ agentic flow
- Agent: "Tôi sẽ tạo OrderValidator struct trong domain/validator.go"
- Human: "Không, đặt ở usecase/validation.go" ← Interrupt sai cách
- Agent: [mất context về plan] → re-plan → inconsistent code



Khi nào interrupt là hợp lý

- Agent đề xuất delete production data
- Agent thêm thư viện không trong approved list
- Agent đang loop > 3 iterations trên cùng error
- Agent đang đi sai direction hoàn toàn

📌 **Quy tắc vàng:** Interrupt = update document, không phải chat correction. Chat correction không persist. Document correction persist.

Pha 4 — Human Review: Duyệt Kế hoạch trước Duyệt Code

Điểm khác biệt quan trọng nhất của ADD so với "just using AI": review xảy ra HAI lần. Lần đầu: review execution plan TRƯỚC khi agent viết dòng code đầu tiên. Lần hai: review code output.

ROI của Plan Review: Review Plan là investment nhỏ với return cực lớn. Một plan sai được phát hiện ở đây = 0 dòng code cần xóa. Một plan sai không được phát hiện = có thể 500 dòng code cần xóa.

Plan Review Prompt + Human Checklist

Plan Review Prompt — Trước khi agent bắt đầu code

Trước khi bắt đầu implement, hãy viết Execution Plan:

1. Files sẽ tạo mới: [list với reasoning]
2. Files sẽ thay đổi: [list với mô tả thay đổi]
3. Files sẽ KHÔNG thay đổi: [confirm scope]
4. Test strategy: [unit tests cho gì, integration tests cho gì]
5. Rủi ro bạn thấy: [potential issues]
6. Assumptions: [những gì bạn assume là đúng]

DỪNG sau khi viết plan. Chờ approval trước khi code.

Human review checklist:

- # ☐ Files list đúng scope (không thêm, không thiếu)?
- # ☐ Test strategy cover acceptance criteria?
- # ☐ Risks được identify đủ?
- # ☐ Assumptions đúng tất cả không?
- # → Nếu YES: "Proceed with implementation"
- # → Nếu NO: correct plan, rồi mới proceed

10.2 — Section 10.2

Prompt Engineering cho Agentic Coding

Prompt cho chatbot và prompt cho agent là hai kỹ năng khác nhau. Chatbot prompt tối ưu cho conversation — ngắn gọn, tự nhiên. Agent prompt tối ưu cho execution — đủ context, rõ ràng về scope, có Definition of Done, và không để lại ambiguity quan trọng.

01	02
10.2.1 — Nguyên tắc cốt lõi: WHAT, không HOW	10.2.2 — Definition of Done (DoD) — Bắt buộc trong Prompt
03	04
10.2.3 — Templates cho 4 task types phổ biến	10.2.4 — Good vs. Bad Prompts — Phân tích chi tiết

10.2.1 — So sánh Prompt types

Nguyên tắc cốt lõi: WHAT, không HOW

Nguyên tắc này tưởng đơn giản nhưng khó thực hành vì instinct của developer thường là "nói với AI cách làm". Trong agentic context, nó cướp đi quyền tự chủ của agent và gán implementation vào approach cụ thể.

Prompt Type	Ví dụ	Vấn đề	Version tốt hơn
HOW-focused (xấu)	"Tao struct OrderValidator với method Validate(order Order) error"	Bạn đang implement, không phải agent	"Implement order validation theo SPEC §3 Functional requirements"
WHAT-focused (tốt)	"Add input validation cho Order creation endpoint"	Tốt nhưng thiếu constraints	"Add input validation cho Order creation endpoint. Xem SPEC.md §3, Constitution §SEC-03"
HOW-focused (xấu)	"Dùng bcrypt với cost 14 để hash password"	Over-specify implementation	"Implement secure password storage cho User registration"
WHAT + DoD (tốt nhất)	"Implement User registration. Done khi: (1) password hashed, (2) tests pass, (3) no linting errors"	✅ WHAT + DoD rõ ràng	← Đây là pattern nên dùng

10.2.2 — Definition of Done

Definition of Done (DoD) — Bắt buộc trong Agent Prompt

DoD là khái niệm quen thuộc trong Agile nhưng ít được áp dụng vào AI prompts. Với agent, DoD đặc biệt quan trọng vì agent không có intuition về "xong là như thế nào" — nếu không được nói rõ, nó có thể dừng khi code compiled, hay khi tests pass một phần.

Definition of Done Template cho Agent Prompts

Task hoàn thành khi tất cả điều kiện sau đều đúng:

Code Quality

- [] go vet không có warnings
- [] golangci-lint không có errors
- [] gofmt không cần format lại (đã formatted)

Tests

- [] Tất cả unit tests pass: go test ./...
- [] Test coverage $\geq$ 80% cho package mới
- [] Không có flaky tests (chạy 3 lần, kết quả nhất quán)

Documentation

- [] Public functions có godoc comment
- [] README được update nếu có breaking change
- [] OpenAPI spec được update nếu có API change

Integration

- [] Không break existing tests
- [] Integration tests pass (nếu relevant): make test-integration

KHÔNG được coi là done nếu:

- Có TODO comments chưa được resolve
- Có panic() calls ngoài main/init functions
- Tests được skip bằng t.Skip()

4 Templates cho Task Types phổ biến

Template 1: New Feature

TEMPLATE: New Feature

Task: Implement [feature name]

Context:

- Spec: .sdd/features/[feature-name]/SPEC.md
- Plan: .sdd/features/[feature-name]/PLAN.md (approved)
- Current task: T[N] từ TASKS.md

Scope:

- Files mới trong: [directories]
- Thay đổi: [specific files nếu biết]
- Không thay đổi: [out of scope files]

Constraints: [Tham chiếu AGENTS.md và Constitution.md]

Definition of Done:

[Paste DoD checklist relevant cho task này]

BƯỚC ĐẦU TIÊN: Viết execution plan.

Chờ approval trước khi bắt đầu implement.

Template 2: Bug Fix

TEMPLATE: Bug Fix

Bug: [mô tả behavior hiện tại]

Expected: [mô tả behavior đúng]

Reproduce: [cách reproduce nếu có]

Impact: [severity + affected users]

Context:

- Bug first seen: [version/date]
- Related spec: [section nếu có]
- Related tests: [test file nếu biết]

Investigation steps (agent tự thực hiện):

1. Đọc error logs/stack trace
2. Tìm root cause (không chỉ fix symptom)
3. Xác định minimal fix (không gold-plate)

Definition of Done:

- [] Root cause được identify và document
- [] Fix không introduce regression
- [] Test case được thêm để prevent recurrence

QUAN TRỌNG: Tìm root cause trước khi fix.

Không acceptable: "add null check" mà không hiểu tại sao null.

10.2.3 — Templates (tiếp)

Template 3: Refactor & Template 4: Test Writing

Template 3: Refactor

TEMPLATE: Refactor

Module cần refactor: [tên module]

Motivation:

- [Performance? Maintainability? Tech debt?]

Goals:

- [Specific improvement 1]

- [Specific improvement 2]

Constraints CỨNG:

- Behavior phải GIỐNG HẾT trước và sau

- Public API không được break

- Tests phải pass trước VÀ sau

Approach:

1. Chạy tests baseline → ghi lại kết quả

2. Refactor incrementally (không big bang)

3. Chạy tests sau mỗi change nhỏ

4. So sánh với baseline trước khi done

Definition of Done:

- [] Behavior unchanged (integration tests pass)

- [] Code complexity giảm

- [] Không có performance regression

KHÔNG acceptable: "refactor" mà thêm features.

Template 4: Test Writing

TEMPLATE: Test Writing

Module cần test: [package/file]

Test type: [unit / integration / e2e]

Coverage target: 80% minimum

Test strategy:

- Happy path: [main success scenarios]

- Error cases: [từ SPEC Unwanted patterns]

- Boundary values: [edge cases từ spec]

- Concurrent scenarios: [nếu relevant]

Test structure:

- File: tests/unit/[package]_test.go

- Naming: TestFunctionName_Scenario_Result

- Helpers: reuse từ tests/helpers/

Definition of Done:

- [] Coverage $\geq$ 80% cho module này

- [] Mỗi EARS requirement có ít nhất 1 test

- [] Tests chạy < 100ms (unit), < 5s (integration)

- [] Không có hardcoded test data (dùng fixtures)

- [] Tests có descriptive names (đọc như spec)

QUAN TRỌNG: Tests phải test BEHAVIOR,
không test IMPLEMENTATION.

10.2.4 — Good vs. Bad Prompts

Good vs. Bad Prompts — Phân tích chi tiết

Chiều	Bad Prompt	Good Prompt
Scope clarity	"Cải thiện order service"	"Add pagination cho GET /orders endpoint theo SPEC §3.2"
HOW vs WHAT	"Dùng cursor-based pagination với id field"	"Implement pagination. Chọn approach phù hợp với current DB schema"
Missing DoD	"Thêm authentication"	"Thêm JWT auth. Done khi: tests pass, constitution compliance, OpenAPI updated"
Missing context	"Fix the bug"	"Fix: GET /orders trả 500 khi user_id là NULL. Xem issue #234, log line 45"
Ambiguous constraints	"Làm nhanh lên"	"Optimize response time cho GET /orders từ 800ms → < 200ms p95"

"Một agent prompt tốt là bản spec thu nhỏ: đủ context để bắt đầu, đủ DoD để biết khi nào dừng, và đủ constraints để không làm sai."

10.3 — Section 10.3

Task-Based Execution Model

Agent hoạt động tốt nhất khi task có scope rõ ràng và context window sạch. Task-Based Execution Model đảm bảo cả hai: mỗi task chạy trong session riêng với context fresh, được tracked trong plan.md.

01	02	03
10.3.1 — Context Pollution — Khi Agent "Ngáo" vì Token Rác	10.3.2 — Plan-Act-Check — Kỹ thuật Self-tracking	10.3.3 — Fresh Session Strategy

"Đây không chỉ là organizational preference — đây là engineering necessity."

10.3.1 — Context Pollution

Context Pollution — Khi Agent "Ngáo" vì Token Rác

Context Pollution xảy ra khi conversation history chứa quá nhiều "noise" — failed attempts, outdated plans, contradicted instructions, abandoned approaches. Agent không phân biệt được "cũ" và "mới" trong context — nó xử lý tất cả như thông tin hiện tại.

Symptoms của Context Pollution

- Agent làm lại những gì đã làm (đã fixed bug nhưng lại add back)
- Agent reference "earlier decision" về thứ đã bị cancel
- Code style không consistent giữa các files trong cùng session
- Agent hỏi lại thông tin đã provide ở đầu session
- Agent đột ngột follow different pattern không có trong CLAUDE.md

Prevention — 3 Rules

- Rule: 1 task = 1 session. Khi task hoàn thành, mở session mới.
- Nếu task cần nhiều sub-steps → chia thành TASKS.md atomic tasks
- Context limit: set hard limit 40-60K tokens per session
- Fresh context = fresh thinking = better output quality

 **Ví dụ thực tế về Context Pollution:** Token 1-5000: "Implement endpoint dùng JSON response" → Token 8001-10000: "À không, cần gRPC thay vì JSON" → Token 18001-25000: Agent cố viết cả JSON lẫn gRPC → Kết quả: agent confused về requirement, mix patterns không consistent.

10.3.2 — Plan-Act-Check

Plan-Act-Check — Kỹ thuật Self-tracking

Plan-Act-Check là pattern cho phép agent tự theo dõi tiến độ và maintain coherence trong một task. Thay vì agent "làm đến đâu nhớ đến đó", nó liên tục update plan.md sau mỗi bước — tạo ra một external memory vững ngoài context window.

```
plan.md — Plan-Act-Check tracking format

# T005: Implement OrderValidator
# Status: IN PROGRESS | Session: 2025-01-20-14h

## Execution Plan (approved 14:02)
- [x] Step 1: Read SPEC.md §3 (OrderValidator requirements)
- [x] Step 2: Create domain/validator.go
- [x] Step 3: Implement ValidateCreateOrder()
- [x] Step 4: Implement ValidateUpdateOrder()
- [ ] Step 5: Write unit tests (tests/unit/validator_test.go)
- [ ] Step 6: Run tests + fix failures
- [ ] Step 7: Run linter

## Current Status
Completed: Steps 1-4. validator.go created with 3 validation rules.
Next: Write tests for ValidateCreateOrder() — 4 test cases needed.

## Issues Encountered
- Step 3: SPEC §3.4 không rõ về max order items.
  Assumption used: max 100 items (theo Constitution §BUS-03).
  → Flagged for human review.

## Failed Attempts (không xóa — giữ lại cho learning)
- Attempt: Dùng custom error type cho validation errors
- Result: Conflicts với existing error handling pattern
- Decision: Use fmt.Errorf with sentinel errors (theo pattern hiện có)

## Rollback Point
Before this task: git commit abc123 (clean state)
```


10.3.3 — Fresh Session Strategy

Fresh Session Strategy

Nguyên lý đơn giản: mỗi TASKS.md atomic task = một session mới. Thêm 30 giây để start session mới rẻ hơn nhiều so với debug context-polluted output.

Scenario	Approach	Lý do
Task mới hoàn toàn	Fresh session với full context	Clean slate, không contamination
Tiếp tục task dở	Fresh session + paste plan.md current state	Reinject state mà không có noise
Debug lỗi nhỏ trong task	Cùng session OK	Issue nhỏ, không accumulate much noise
Đã loop > 3 lần	Fresh session + revised approach	Context pollution rõ ràng, cần restart
Session > 1 giờ hoặc > 50K tokens	Summarize + fresh session	Prevent cliff effect

Fresh Session Context Injection Script

```
#!/bin/bash
# scripts/prep_agent_context.sh
echo "=== AGENT CONTEXT FOR NEW SESSION ==="
echo "## Project Identity"
cat .sdd/AGENTS.md | head -30
echo "## Current Task State"
cat plan.md | grep -A5 "IN PROGRESS"
echo "## Recent Decisions"
git log --oneline -5
echo "## Test Status"
go test ./... 2>&1 | tail -5
```

10.4 — Section 10.4

Constraint Documents — Giữ Agent trong Khuôn khổ

Constraint documents là "hàng rào" của agent. Thiếu constraint = agent "tự chế" mọi thứ: chọn thư viện không được approve, đặt tên theo convention nó học từ training data, tạo cấu trúc file không khớp với project, hay implement security theo cách nó thấy phổ biến chứ không phải cách bạn cần.

"Điều thú vị: thiếu constraint không chỉ tạo ra code xấu mà còn tạo ra code nguy hiểm. Agent không cố ý xấu khi dùng thư viện có vulnerability — nó chỉ không biết thư viện đó không được approved."



10.4.1 — Constraint Hierarchy

Template Constraint Hierarchy — 3 Layers

Constraints được tổ chức theo ba tầng: Global (toàn project), Business (nghịệp vụ), và Safety (an toàn). Mỗi tầng có mức độ enforcement khác nhau và được document trong file riêng.

CONSTRAINT HIERARCHY

LAYER 1: GLOBAL CONSTRAINTS (.sdd/constraints/global.md)

Tech Stack: Go 1.23+, PostgreSQL 16, Redis 7

Banned libs: github.com/jinzhu/gorm (use pgx + sq)

File naming: snake_case, plurals for packages

Test pattern: testify/require, not assert

Enforcement: Auto (golangci-lint catches violations)

LAYER 2: BUSINESS CONSTRAINTS (.sdd/constraints/business.md)

Auth: Không lưu password dạng plaintext. Dùng argon2id.

API: Mọi endpoint phải có Rate Limit header

Data: Soft delete only cho user/order data

Audit: Mọi write operation cần audit log entry

Enforcement: Code review + CI checks

LAYER 3: SAFETY CONSTRAINTS (.sdd/constraints/safety.md)

KHÔNG tự ý xóa dữ liệu trong /data/ mà không confirm

KHÔNG commit vào production branches

KHÔNG thêm go.mod dependency mà không hỏi

Enforcement: Human approval gate + CI block

Global Constraints — Stack và Conventions

global.md — Tech Stack + Naming + Approved Packages

TECHNOLOGY STACK (immutable trừ khi có RFC)

Backend

Language: Go 1.23+

HTTP: net/http + chi router (v5)

Database: pgx/v5 (NOT gorm, NOT sqlc direct — dùng sq query builder)

Cache: go-redis/v9

Events: segmentio/kafka-go

Testing: testify/suite + testify/mock

Container: Docker (multi-stage builds)

NAMING CONVENTIONS

Packages: lowercase, plural, no underscores (orders/, not order/)

Files: snake_case (order_validator.go, not orderValidator.go)

Interfaces: Er suffix (OrderStorer, not IOrderStore)

Errors: Err prefix (ErrOrderNotFound, not NotFoundError)

Constants: SCREAMING_SNAKE for env vars, CamelCase for Go consts

APPROVED EXTERNAL PACKAGES

github.com/go-chi/chi/v5 # HTTP router

github.com/jackc/pgx/v5 # PostgreSQL driver

github.com/redis/go-redis/v9 # Redis client

github.com/Masterminds/squirrel # SQL query builder

github.com/stretchr/testify # Testing

go.uber.org/zap # Structured logging

BANNED PACKAGES (với lý do)

github.com/jinzhu/gorm # Performance issues, magic behavior

github.com/gorilla/mux # Replaced by chi, inconsistent API

github.com/dgrijalva/jwt-go # Known vulnerabilities, archived

ADDING NEW PACKAGES

Quy trình: PR với justification → tech lead approve → update this file.

Agent KHÔNG được add package mà không có approval.

10.4.3 — Business Constraints

Business Constraints — Rules nghiệp vụ + Domain Glossary

Authentication & JWT Rules

- Hash algorithm: argon2id (memory: 64MB, time: 3, threads: 4)
- KHÔNG dùng: bcrypt, md5, sha1, sha256 cho passwords
- JWT: RS256 (asymmetric — không HS256)
- Access token TTL: 15 phút
- Refresh token TTL: 7 ngày, single-use, rotate on refresh
- Claims phải có: sub, iat, exp, jti (unique ID for revocation)

PII Data + Soft Delete Rules

- Phone: log dưới dạng "0912***456" (mask 3 digits)
- Email: log dưới dạng "use***@domain.com" (mask 3 chars)
- Không bao giờ log: password, payment card, national ID
- Soft Delete: business entities dùng deleted_at timestamp (NOT hard delete)
- Hard delete chỉ cho: logs > 90d, temp files, test data

Domain Glossary

- **Order:** Confirmed purchase intent. Has order items.
- **Cart:** Unconfirmed items. Can be abandoned.
- **Invoice:** Financial document for completed order.
- **Fulfillment:** Process from payment to delivery.
- **Tenant:** B2B customer (company using platform).
- **User:** End user (employee of a Tenant).

10.4.4 — Safety Constraints

Safety Constraints — Guardrails cho Agent

KHÔNG ĐƯỢC (cần human confirm)

- Xóa bất cứ thứ gì trong /data/ directory
- DROP TABLE, TRUNCATE trong migration files
- DELETE FROM ... WHERE không có WHERE clause
- Thay đổi column type của existing data (migration risk)
- Thêm package vào go.mod (hỏi trước)
- Modify .github/workflows/ (CI changes = security sensitive)
- Thay đổi Docker base image (security review needed)
- Push vào main/master/production branches

PHẢI LÀM (mandatory)

- Tạo git checkpoint trước mọi schema migration
- Test migration với rollback plan
- Backup reminder trước migration
- Không access production database trực tiếp
- Không hardcode production endpoints, credentials
- Không bypass auth middleware "cho nhanh"
- Khi không chắc chắn: dừng lại và báo cáo
- "Tôi không chắc về constraint X. Làm thế nào bạn muốn?"

⚠ **Tại sao thiếu Constraints = Agent "tự chế":** Ví dụ: thiếu naming convention → agent dùng camelCase vì training data nhiều JS. Thiếu library constraint → agent dùng GORM vì phổ biến trong Go tutorials. Thiếu auth constraint → agent dùng bcrypt vì "secure enough" theo docs. Constraint docs = "your reasonable" được document cho agent.

10.4.5 — Verify Constraint Adherence

Verify Constraint Adherence

Post-implementation Constraint Check Prompt

Prompt: Verify constraint compliance sau khi agent submit code

Review code bạn vừa viết và xác nhận compliance với constraints:

Global Constraints check:

- ☐ Có dùng library nào ngoài approved list?
- ☐ Naming conventions đúng (snake_case files, plural packages)?
- ☐ Error wrapping format đúng: `fmt.Errorf("context: %w", err)`?

Business Constraints check:

- ☐ Authentication code dùng argon2id?
- ☐ API endpoints có rate limit headers?
- ☐ Soft delete được implement (không hard delete)?
- ☐ PII data không xuất hiện trong logs?

Safety Constraints check:

- ☐ Không DELETE mà không có WHERE clause?
- ☐ Không hardcode credentials hoặc production URLs?
- ☐ Không bypass auth middleware?

Format: Liệt kê mỗi check với

10.5 — Section 10.5

Hands-on Lab — Có vs. Không có ADD

Đây là bài lab đối chứng được thiết kế để cho bạn thấy sự khác biệt giữa "dùng AI theo bản năng" và "dùng AI theo ADD workflow" qua một feature thực tế: User Authentication. Cùng một feature, cùng một AI tool — kết quả khác nhau rõ rệt.



Setup trước khi bắt đầu Lab:

Cài đặt: Cursor (Agent Mode) hoặc Cline trong VSCode

API key: Mua qua Cline (model: claude-sonnet-4-5)

Starter project: git clone <https://github.com/your-org/add-lab-starter>

Hoặc tạo fresh: mkdir auth-lab && cd auth-lab && go mod init auth-lab

Time estimate: Round 1 ~15 phút, Round 2 ~45 phút

Yêu cầu: Không sửa code tay trong cả hai rounds — chỉ prompt agent

10.5.1 — Round 1

Round 1 — Không có ADD (Vague Intent)

Trong Round 1, bạn sẽ cố tình làm sai: đưa một intent mơ hồ cho agent và để nó tự quyết định mọi thứ. Không có AGENTS.md, không có CLAUDE.md, không có constraints, không có spec. Chỉ có một câu prompt.

Bước 1: Minimal Setup

Round 1 setup

```
mkdir -p auth-lab/{cmd,internal}  
cd auth-lab  
go mod init github.com/lab/auth
```

Không tạo: AGENTS.md, CLAUDE.md, constraints
Không configure: naming, library choices

Bước 2: Vague Prompt

Round 1 prompt

Build me a user authentication system
for a Go web app.
Include registration and login.

Đây là tất cả. Không thêm gì.
Để agent tự quyết định mọi thứ.

 **Dự đoán Round 1 outcome:** Agent thường chọn: gorilla/mux hoặc gin (không phải chi) • Password: bcrypt (không phải argon2id theo business constraint) • JWT: dgrijalva/jwt-go (package có known CVE, archived) • Structure: sẽ không match Clean Architecture của project • Rate limiting: thường không có • Validation: thường ad-hoc, không consistent

Round 1 — Quan sát và Ghi nhận

Quan sát	Agent chọn (Round 1)	Ghi chú của bạn
HTTP framework	(quan sát và ghi lại)	Thường: gorilla/mux hoặc gin
Password hashing library	(quan sát và ghi lại)	Thường: bcrypt (không phải argon2id)
JWT library	(quan sát và ghi lại)	Có thể: dgrijalva/jwt-go (deprecated!)
File/folder structure	(quan sát và ghi lại)	Thường không match Clean Architecture
Error handling style	(quan sát và ghi lại)	Có thể không dùng fmt.Errorf wrap
Testing included?	Yes / No	Thường: có nhưng basic
OpenAPI docs?	Yes / No	Thường: không có
Rate limiting?	Yes / No	Thường: không có
Validation library	(quan sát và ghi lại)	Thường: ad-hoc, không consistent
Database ORM/driver	(quan sát và ghi lại)	Thường: gorm (banned trong project)

Round 1 evaluation commands

```
# Kiểm tra code quality
go vet ./...
golangci-lint run 2>/dev/null || echo "No linter configured"

# Kiểm tra tests
go test ./... 2>&1 | tail -10

# Đếm lines of code
find . -name "*.go" | xargs wc -l | tail -1

# Check dependencies được thêm vào
cat go.sum | wc -l
```

Round 2 — Với ADD Workflow (60 phút)

Trong Round 2, bạn áp dụng đầy đủ ADD workflow: Context Setup, defined Intent với DoD, Plan Review, và structured execution. Cùng một feature — authentication — nhưng với proper setup.

Pha 1: Context Setup (15 phút)

- Tạo AGENTS.md với Senior Go developer persona
- Tạo CLAUDE.md với project context, Clean Architecture
- Tạo business constraints: argon2id, RS256 JWT, rate limit
- Định nghĩa file structure: /cmd, /internal/{domain,usecase,handler,infra}

Pha 2: SPEC.md cho feature

- Registration: email + password → argon2id hash → store → 201 với user_id
- Login: email + password → verify → JWT RS256 (15min) + refresh (7d)
- Error: 409 conflict, 401 với same message (prevent user enumeration)
- Out of scope: email verification, OAuth, 2FA, password reset

Pha 3 & 4: Plan → Execute với DoD

- Agent viết Shadow Plan trước khi execute
- Human review: scope đúng? File list đúng? Assumptions đúng?
- Execute với DoD: go vet pass, golangci-lint clean, tests ≥ 80%, no passwords in logs, no banned packages

Round 2 setup script (rút gọn)

```
# Tạo context documents
mkdir -p .sdd/{constraints,features/feat-auth}

# 1) AGENTS.md
cat > AGENTS.md

# 2) CLAUDE.md
cat > CLAUDE.md

# 3) Business constraints
cat > .sdd/constraints/business.md

SPEC + Plan Review + Execute

# SPEC cho feature
cat > .sdd/features/feat-auth/SPEC.md

# Prompt Plan review:
# Đọc AGENTS.md, CLAUDE.md, business.md, SPEC.md
# Viết plan: files tạo mới/thay đổi, dependencies, test strategy, risks
# DỪNG sau plan và chờ approval

# Prompt Execute:
# Implement theo plan approved + DoD + Shadow Plan trước khi chạy
```

10.5 — So sánh kết quả

Bước 5: So sánh Round 1 vs Round 2

Tiêu chí	Round 1 (No ADD)	Round 2 (With ADD)	Winner
Library choices	Agent chọn tự do	Theo approved stack	R2 (predictable)
Password algorithm	Thường bcrypt	argon2id (per spec)	R2 (spec compliance)
JWT library	Có thể deprecated	golang-jwt (approved)	R2 (security)
File structure	Ad-hoc	Clean Architecture	R2 (maintainable)
Tests present	Maybe	Yes (per DoD)	R2 (verifiable)
go vet pass	Maybe	Yes (per DoD)	R2 (quality)
Rate limiting	Likely missing	Per business.md	R2 (complete)
User enumeration	Likely vulnerable	Protected (per spec)	R2 (secure)
Time to first line of code	Faster	Slower (15 min setup)	R1 (speed)
Rework needed	High	Low	R2 (efficiency)

10.5 — Key Insight

Key Insight từ Comparison Lab

"ADD không làm bạn code chậm hơn.
Nó làm bạn rework ít hơn."

15'

Setup R2

Setup R2 → code đúng ngay từ đầu

2-3×

Rework R1

Rework R1 mất gấp 2-3× thời gian viết

5+

ADD ROI

ADD ROI dương từ feature >5 requirements

Round 1 nhanh hơn để bắt đầu nhưng tạo ra "fast path to wrong destination". Round 2 đầu tư 15 phút setup → code đúng specification ngay từ đầu. Rework của Round 1 thường mất 2-3× thời gian initial writing. Key takeaway: ADD không làm bạn code chậm hơn. Nó làm bạn rework ít hơn.

10.5.3 — Reflection Questions

Câu hỏi Reflection

Q1 — So sánh lựa chọn của Agent

Trong Round 1, agent chọn những gì bạn đã dự đoán không? Nếu không, tại sao? Điều đó nói lên gì về khoảng cách giữa "obvious choice" của agent và "obvious choice" của team bạn?

Q2 — Hidden Rework Cost

Nếu bạn phải deploy Round 1 code lên production: có bao nhiêu thứ cần sửa trước? List ra. Đây là "hidden rework cost" của prompt-only approach.

Q3 — Round 2 Tradeoffs

Với Round 2: phần nào trong setup mất nhiều thời gian nhất? AGENTS.md, SPEC.md, hay constraints? Có phần nào bạn nghĩ có thể skip mà không ảnh hưởng nhiều đến output?

Q4 — ROI cho dự án của bạn

Cho dự án cụ thể của bạn hiện tại: ADD would pay off từ feature/sprint size nào? Tính ROI cụ thể theo công thức: $\text{setup_time} + \text{review_time}$ vs $\text{expected_rework_time_without_ADD}$.

10.6 — Section 10.6

Kỹ thuật Nâng cao — Shadowing + Token Management

Hai kỹ thuật trong section này giải quyết hai vấn đề thực tế quan trọng: (1) tốn tiền API vì agent hiểu sai intent và chạy nhiều vòng thừa, và (2) context window bị chiếm bởi files không liên quan làm giảm chất lượng reasoning. Đơn giản nhưng có impact lớn.

01

10.6.1 — Kỹ thuật "Shadowing" — Plan trước khi Execute

02

10.6.2 — Token Management — Tối ưu Context Window

03

10.6.3 — .gitignore cho Agent — Context hygiene

Kỹ thuật "Shadowing" — Plan trước khi Execute

Shadowing là kỹ thuật yêu cầu agent "nói to lên kế hoạch" trước khi bắt đầu làm. Tên "shadowing" đến từ shadow boxing — "đánh vào không khí" trước, cho bạn thấy nó sẽ làm gì trước khi nó thực sự hit production code.

ROI thực tế của Shadowing: Nếu agent hiểu sai intent và bạn phát hiện ở bước plan → bạn tiết kiệm toàn bộ token của execution. Với mỗi session trung bình 20,000–50,000 tokens, phát hiện intent sai sớm có thể tiết kiệm \$0.05–\$0.15 per session — nhỏ nhưng tích lũy đáng kể.

Shadowing Prompt Template — Bắt buộc thêm vào mọi agent task

"TRƯỚC KHI thực hiện bất kỳ action nào (read file, write, execute):

Viết SHADOW PLAN:

- 1. Tôi hiểu task này là: [một câu tóm tắt]
- 2. Approach của tôi: [3-5 bullet points, mỗi point = 1 action]
- 3. Files tôi sẽ READ: [list]
- 4. Files tôi sẽ CREATE/MODIFY: [list]
- 5. Commands tôi sẽ RUN: [list]
- 6. Expected outcome: [mô tả kết quả mong đợi]

DỪNG và hiển thị shadow plan. Đợi 'proceed' trước khi làm gì."

- # Human review shadow plan:
- # - Scope đúng không?
- # - Có file nào không cần modify bị đưa vào list?
- # - Expected outcome có match intent không?
- # → "Proceed" → Agent bắt đầu execute
- # → "Adjust: [correction]" → Agent revise plan, shadow lại

Ví dụ output Shadowing

```
# === VÍ DỤ OUTPUT ===
SHADOW PLAN:
1. Task: Thêm email validation cho User registration
2. Approach:
  - Đọc SPEC.md để hiểu validation rules
  - Đọc domain/user.go để hiểu User struct
  - Thêm validate_email() trong domain/validators.go
  - Update RegisterUser usecase để call validation
  - Viết tests cho happy path + invalid formats
3. Files READ: SPEC.md, domain/user.go, usecase/register.go
4. Files MODIFY: domain/validators.go, usecase/register.go
   Files CREATE: tests/unit/validator_test.go
5. Commands: go test ./domain/..., golangci-lint run
6. Expected: Email validation working, tests green, linter clean
```


10.6.1 — So sánh Techniques

Shadowing vs Clarification-First (Chương 7.2)

Shadowing và Clarification-First (7.2) nghe tương tự nhưng phục vụ mục đích khác nhau:

	Clarification-First (7.2)	Shadowing (10.6)
Khi dùng	Khi spec còn ambiguous	Khi spec đã clear, task đã defined
Mục đích	Tìm điểm mờ trong spec	Verify agent hiểu đúng task
Output	Questions + assumptions	Execution plan + file list
Timing	Trước khi viết/lock spec	Trước mỗi execution session
Cost saving	Saves spec rewrite cost	Saves execution token cost

"Shadowing là 'insurance policy' trước mỗi execution — chi phí thấp (vài tokens), return cao (tránh wrong execution hoàn toàn)."

Clarification-First dùng khi spec còn mơ hồ; Shadowing dùng khi spec đã rõ và chuẩn bị execute. Hai kỹ thuật bổ sung nhau, không thay thế nhau.

10.6.2 — Token Management

Token Management — Tối ưu Context Window

Context window là tài nguyên hữu hạn và tốn tiền. Khi context đầy bởi những thứ không relevant (node_modules, git history, build artifacts), agent có ít "space" hơn để xử lý những gì thực sự quan trọng.

Context Budget Planning — Task "Implement OrderValidator"

STATIC (load once per session)

AGENTS.md: ~800 tokens

CLAUDE.md: ~1500 tokens

Constraint docs (3): ~2000 tokens

Current SPEC.md: ~2000 tokens

PLAN.md + TASKS.md: ~1000 tokens

Static total: ~7,300 tokens

DYNAMIC (accumulates during session)

Per source file read: ~2,000 tokens avg

Per command output: ~500 tokens avg

Per thinking block: ~1,500 tokens avg

Per generated code: ~2,000 tokens avg

Total estimate: ~25,300 tokens

Cost: $25,300 \times \$3/M + 6,000 \times \$15/M = \sim \$0.17$

Set hard limit: "cline.maxTokensPerTask": 35000 (40% buffer trên estimate)

Nếu task exceed 35K tokens → stop, check nếu bị loop

Kỹ thuật tiết kiệm context

- Summarize-before-continue: ở 30K tokens, viết 500-token summary rồi fresh session
- Read selectively: chỉ đọc file relevant cho task hiện tại
- Targeted grep: "tìm tất cả nơi X được dùng" thay vì đọc mọi file

Model selection strategy

- Haiku: cho read/search operations (rẻ hơn)
- Sonnet: chỉ khi cần reasoning phức tạp
- Incremental context: inject thêm context chỉ khi agent cần, không dump toàn bộ ngay từ đầu

.gitignore cho Agent — Context Hygiene

.agentignore — Exclude files from agent context

```
# Build artifacts
dist/
build/
*.exe
*.dll
*.so

# Dependencies
node_modules/
vendor/ # Go vendor directory

# Version control
.git/
.gitignore

# IDE config
.vscode/
.idea/
*.swp

# Logs và debug
*.log
logs/
tmp/
coverage/

# Large generated files
*.pb.go
*_gen.go
docs/swagger.json

# Binary files
*.png
*.jpg
*.gif
*.ico
*.wasm

# Sensitive
.env
.env.*
*.pem
*.key
*.secret
secrets/
```

Cline context config (.vscode/settings.json)

```
{
  "cline.respectGitignore": true,
  "cline.excludePatterns": [
    "**/node_modules/**",
    "**/.git/**",
    "**/vendor/**",
    "**/dist/**",
    "**/*.log",
    "**/.env*"
  ],
  "cline.alwaysAllowPatterns": [
    "**/*.go",
    "**/*.md",
    "**/*.yaml",
    "**/*.json"
  ]
}
```

 **Rule of thumb:**

Include: source files, specs, tests, config

Exclude: generated files, deps, build artifacts, secrets

Selective: large generated files (proto, swagger) only when needed

Tổng kết

Tổng kết Chương 10

Chương này đã xây dựng Agent-Driven Development như một workflow hoàn chỉnh: từ Context Setup (định nghĩa Agent là ai) qua Intent Communication (WHAT không HOW) đến Agentic Execution (Plan-Act-Check loop) và Human Review (duyet plan trước code). Cùng với đó là ba công cụ thực hành: Task-Based Execution, Constraint Documents, và kỹ thuật Shadowing để tiết kiệm token.

Section	Key Technique	Điểm cốt lõi
10.1 — ADD Pipeline	4-phase workflow	Pha 1 (Context) quan trọng nhất; Pha 4 review Plan trước Code
10.2 — Prompt Engineering	WHAT + DoD template	Mô tả outcome, không describe implementation
10.3 — Task Execution	Plan-Act-Check + Fresh Session	Context pollution là kỹ thuật vấn đề, không phải subjective
10.4 — Constraints	3-layer hierarchy	Thiếu constraint = agent chọn "training data default"
10.5 — Lab	Round 1 vs Round 2	Trải nghiệm trực tiếp mạnh hơn mọi lời giải thích
10.6 — Advanced	Shadowing + Token mgmt	Shadow plan tiết kiệm execution cost khi intent mơ hồ

Key Takeaways

5 Takeaways quan trọng nhất

1 Pha 1 là investment, không phải overhead

Mỗi phút thiết lập AGENTS.md và CLAUDE.md tiết kiệm 10 phút debug sau. "Thiết lập tốt ở Pha 1 = ít can thiệp ở Pha 3."

2 WHAT + DoD, không phải HOW

Mô tả outcome và DoD, để agent tự chọn HOW trong khuôn khổ constraints. Chatbot prompt ≠ Agent prompt.

3 1 task = 1 session

Context pollution là engineering problem, không phải subjective preference. Fresh context = fresh thinking = better output.

4 Constraint docs = "your reasonable" cho agent

Agent không lazy. Thiếu constraint → agent dùng "training data best practice" — reasonable nhưng không phải của team bạn.

5 ADD không làm chậm — nó giảm rework

Round 1 nhanh hơn bắt đầu nhưng rework cao. Round 2 setup 15 phút → code đúng ngay từ đầu. ROI dương từ feature > 5 requirements.

Kết thúc Chương 10

Chương 10 hoàn thành

"ADD không thay thế SDD — chúng hoạt động cùng nhau.
SDD cung cấp Spec → ADD cung cấp Execution Workflow."

📌 Recap ADD Pipeline — 4 Pha:

Pha 1: Context Setup (AGENTS.md + CLAUDE.md + Constraints) → Pha 2: Intent Communication (WHAT + DoD) → Pha 3: Agentic Execution (Plan → Code → Test loop) → Pha 4: Human Review (Plan trước Code)

Playbook SDD + ADD — © 2025

Agent-Driven Workflow

Chương 10 — Hoàn thành